Simon Sutoris 7542170

1 Simply Typed Lambda Calculus

$$\dfrac{\dfrac{\dfrac{x : \mathbf{A} \in y : \mathbf{A}, x : \mathbf{A}}{y : \mathbf{A}, x : \mathbf{A} \vdash x : \mathbf{A}}\ \textsc{Ty-Var}}{y : \mathbf{A} \vdash (\lambda x : \mathbf{A}.x) : \mathbf{A} \to \mathbf{A}}\ \textsc{Ty-Abs} \qquad \dfrac{y : \mathbf{A} \in y : \mathbf{A}}{y : \mathbf{A} \vdash y : \mathbf{A}}\ \textsc{Ty-Var}}{y : \mathbf{A} \vdash ((\lambda x : \mathbf{A}.x)\, y) : \mathbf{A}}\ \textsc{Ty-App}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{y : \mathbf{C} \in x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{B}, w : \mathbf{A}}{x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{B}, w : \mathbf{A} \vdash y : \mathbf{C}}\ \textsc{Ty-Var}}{x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{B} \vdash (\lambda w : \mathbf{A}.y) : \mathbf{A} \to \mathbf{C}}\ \textsc{Ty-Abs}}{x : \mathbf{B}, y : \mathbf{C} \vdash (\lambda z : \mathbf{B}.\lambda w : \mathbf{A}.y) : \mathbf{B} \to (\mathbf{A} \to \mathbf{C})}\ \textsc{Ty-Abs} \qquad \dfrac{x : \mathbf{B} \in x : \mathbf{B}, y : \mathbf{C}}{x : \mathbf{B}, y : \mathbf{C} \vdash x : \mathbf{B}}\ \textsc{Ty-Var}}{x : \mathbf{B}, y : \mathbf{C} \vdash ((\lambda z : \mathbf{B}.\lambda w : \mathbf{A}.y)\, x) : \mathbf{A} \to \mathbf{C}}\ \textsc{Ty-App}$$

There exist no context Γ or type T for the two latter terms t in the exercise. For the first, w is a free variable, and we can't deduce its type without evaluating it: Or in other words: There is no typing rule that lets us determine its type.

For the latter: x must be of a type, that is both a concrete type, and can be evaluated. Both can't be simultaneously true.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{z : \mathbf{A} \in \Gamma, x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{A}}{\Gamma, x : \mathbf{B}, y : \mathbf{C}, z : \mathbf{A} \vdash z : \mathbf{A}}\ \textsc{Ty-Var}}{\Gamma, x : \mathbf{B}, y : \mathbf{C} \vdash (\lambda z : \mathbf{A}.z) : \mathbf{A} \to \mathbf{A}}\ \textsc{Ty-Abs} \qquad \dfrac{\lightning}{\Gamma, x : \mathbf{B}, y : \mathbf{C} \vdash (w\, x\, y) : \mathbf{A}}}{\Gamma, x : \mathbf{B}, y : \mathbf{C} \vdash ((\lambda z : \mathbf{A}.z)\, (w\, x\, y)) : \mathbf{A}}\ \textsc{Ty-App}}{\Gamma, x : \mathbf{B} \vdash \lambda y : \mathbf{C}.((\lambda z : \mathbf{A}.z)\, (w\, x\, y)) : \mathbf{C} \to \mathbf{A}}\ \textsc{Ty-Abs}}{\Gamma \vdash \lambda x : \mathbf{B}.\lambda y : \mathbf{C}.((\lambda z : \mathbf{A}.z)\, (w\, x\, y)) : \mathbf{B} \to (\mathbf{C} \to \mathbf{A})}\ \textsc{Ty-Abs}$$

$$\dfrac{\dfrac{\dfrac{\lightning}{x : \tau', y : \mathbf{A} \vdash x : (\tau \to \tau')} \qquad \dfrac{x : \tau' \in x : \tau', y : \mathbf{A}}{x : \tau', y : \mathbf{A} \vdash x : \tau'}\ \textsc{Ty-Var}}{x : \tau', y : \mathbf{A} \vdash x\, x : \tau'}\ \textsc{Ty-App}}{x : \tau* \vdash \lambda y : \mathbf{A}.x\, x : \mathbf{A} \to \tau'}\ \textsc{Ty-Abs}$$

2 Extension with Numbers

Theorem 1. *Preservation of Types under Substitution (9.3.8)*

$$\dfrac{\Gamma, x : S \vdash t : T \qquad \Gamma \vdash s : S}{\Gamma \vdash t[s/x] : T}$$

Proof. Case T-Var:
If $t = x$, then $x : S \to x[s/x] = s : S \Rightarrow t$ and $t[s/x]$ have the same type S.
If $t \neq x$, then $t \to t[s/x] : T = t : T$.
Case T-Zero:
$[s/x]t = \texttt{zero} : \texttt{Nat}$
Case T-Abs:
TODO
Case T-App:
TODO
Case T-Succ:
If $t = x$, then $S = \texttt{Nat} \Rightarrow x : \texttt{Nat} \to x[s/x] = s : \texttt{Nat}$.
If $t \neq x$, then $t[s/x] = t : \texttt{Nat}$

□

Theorem 2. *Progress (9.3.5)*
Let t be a closed, well-typed term, that is $\vdash t : T$ for some T, then either t is a value or there is some t' with $t \to t'$.

Proof. T-Var does not apply, because not closed. T-Zero, T-Abs, T-Succ are values by definition.

For T-App, because of the rules axioms $\Gamma \vdash t_1 : T_{11} \to T_{12}$ and $\Gamma \vdash t_2 : T_{11}$, it follows that $t_1\, t_2 : T_{12} \to [x/t_2]t_1 : T_{12}$. □

Theorem 3. *Preservation (9.3.9)*
If $\Gamma \vdash t : T$ and $t \to t'$, then $\Gamma \vdash t' : T$.

Proof. T-Var, T-Zero and T-Abs don't apply, because there are no derivation rules for them.
Case T-Succ:

$$t : \texttt{Nat} \tag{1}$$
$$t' = \texttt{succ}\, t_1 : \texttt{Nat} \tag{2}$$

Because t and t' have the same type `Nat` and $t \to t'$, E-Succ applies.
Case T-App $t_1 \to t_1'$:
TODO
Case T-App $t_1 = v_1 \wedge t_2 \to t_2'$:
TODO
Case T-App $t_1 = (\lambda x : T.t_{12}) \wedge t_2 = v_2$:
TODO

□

3 Environments and free variables

Exercise 1 follows from Exercise 2. Therefore I first prove Exercise 2 (Theorem 4) and afterwards Exercise 1 (Theorem 5).

Theorem 4. *If $\Gamma \vdash t : T$, then $FV(t) \subseteq dom(\Gamma)$.*

Proof. Case T-Var
If $t = x$, then $FV(x) = \{x\}$ and $x \in dom(\Gamma)$. Therefore $FV(t) \subseteq dom(\Gamma)$.
Case T-Abs
If $t = \lambda x : S.u$, then $FV(t) = FV(u) \setminus \{x\}$.
Apply theorem hypothesis to $u \Rightarrow FV(u) \subseteq dom(\Gamma) \cup \{x\}$.
Therefore: $FV(t) = FV(u) \setminus \{x\} \subseteq (dom(\Gamma) \cup \{x\}) \setminus \{x\} = dom(\Gamma)$.
Case T-App
Very similar to Case T-Abs.
If $t = t_1\, t_2$, then $FV(t) = FV(t_1) \cup FV(t_2)$.
If $t_1 = \lambda x : S.u_1$, then $FV(t) = FV(u_1) \setminus \{x\}$.
Apply theorem hypothesis to $u_1 \Rightarrow FV(u_1) \subseteq dom(\Gamma) \cup \{x\}$.
Therefore: $FV(t_1) = FV(u_1) \setminus \{x\} \subseteq (dom(\Gamma) \cup \{x\}) \setminus \{x\} = dom(\Gamma)$.
Analog for $t_2 = \lambda x : S.u_2$.
Therefore: $FV(t) = FV(t_1) \cup FV(t_2) \subseteq dom(\Gamma) \cup dom(\Gamma) = dom(\Gamma)$

□

Theorem 5. *If $\Gamma \vdash t : T$, and $x \notin dom(\Gamma)$, then for all values v we have $t[v/x] = t$.*

Proof. Position Theorem 4 : $v \in FV(t) \to v \in dom(\Gamma)$
Counterposition: $v \notin dom(\Gamma) \to v \notin FV(t)$
Therefore: $\forall v, t[v/x] = t$

□

4 Compilation to untyped lambda calculus

Theorem 6. *$[\![v]\!]$ is a value.*

Proof. Case $[\![x]\!]$ and x is a value
$[\![x]\!] = x$. Therefore $[\![x]\!]$ is a value.
Case $[\![\lambda x : T.t]\!]$
If t is a value, then $[\![\lambda x : T.t]\!] := \lambda x.[\![t]\!] = t$ is a value.
If t is not a value, then by theorem hypothesis $[\![t]\!]$ is a value.
Therefore $\lambda x.[\![t]\!]$ is in the form $\lambda x.t$ (Figure 5.3 in the Book), and thus a value.
Case $[\![\texttt{true}]\!]$
$[\![\texttt{true}]\!] := \lambda x.\lambda y.x$ is in the form $\lambda x.t$ (Figure 5.3 in the Book) and is therefore a value.
Case $[\![\texttt{false}]\!]$
Analog to case $[\![\texttt{true}]\!]$
Case $[\![t_1\ t_2]\!]$, $[\![\texttt{if}\ t_1\ \texttt{then}\ t_2\ \texttt{else}\ t_3]\!]$
The terms are not in the form $[\![v]\!]$ and thus the theorem does not apply.

□

Theorem 7. *If $t \to v$ in the calculus above, then $[\![t]\!] \to^* [\![v]\!]$ in the untyped call-by-value lambda calculus.*

Proof. Case $[\![x]\!]$, $[\![\lambda x : T.t]\!]$, $[\![\texttt{true}]\!]$, $[\![\texttt{false}]\!]$
As per Theorem 6, the terms are values, and thus Theorem 7 does not apply.
Case $[\![t_1\ t_2]\!]$
Due to call-by-value and the theorem hypothesis, $[\![t_1]\!] \to^* [\![v_1]\!]$
$[\![v_1\ t_2]\!] \to [\![v_1[t_2/x]]\!]$
Due to the *progress* theorem, $v_1[t_2/x]$ either is a value, or $\to^*$ reduces it to a value.
Therefore $[\![v_1\ t_2]\!] \to^* [\![v]\!]$
$[\![v_1\ t_2]\!] := [\![v_1]\!]\ [\![t_2]\!]$
Then $[\![v_1]\!]\ [\![t_2]\!] \to [\![v_1]\!][[\![t_2]\!]/x]$
Then the exercise allows us to assume $[\![v_1]\!][[\![t_2]\!]/x] = [\![v_1[t_2/x]]\!]$
This reproduces the case above and we get $[\![v_1\ t_2]\!] := [\![v_1]\!]\ [\![t_2]\!] \to^* [\![v]\!]$
Case $[\![\texttt{if}\ t_1\ \texttt{then}\ t_2\ \texttt{else}\ t_3]\!]$
If $t_1 = \texttt{true}$, then E-IfTrue applies:
$[\![\texttt{if}\ t_1\ \texttt{then}\ t_2\ \texttt{else}\ t_3]\!] \to [\![t_2]\!]$
As per theorem hypothesis, $t_2 \to^* v$.
If $t_1 = \texttt{false}$, then E-IfFalse applies. Analog to $t_1 = \texttt{true}$.
Assuming a typo in the exercise, E-If is actually

$$\frac{t_1 \to t_1'}{\texttt{if}\ t_1\ \texttt{then}\ t_2\ \texttt{else}\ t_3 \to \texttt{if}\ t_1'\ \texttt{then}\ t_2\ \texttt{else}\ t_3}$$

Then, due to the *preservation* theorem, $t_1 : T \to^* t_1' : T$, t_1 eventually reduces to `true` or `false`.
Then the cases above apply.

□

5 Types and Programming Languages

No questions.